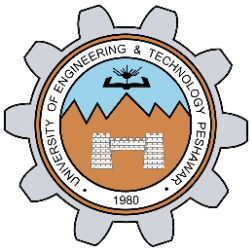
A Taste of Data path + Control Design Example:

Factorial Circuit (Open Ended Lab)

LAB # 11



Spring 2024

CSE-308L Digital System Design Lab

Submitted by: **Ali Asghar**

Registration No.: **21PWCSE2059**

Class Section: **C**

"On my honor, as student of University of Engineering and Technology, I have neither given nor received unauthorized assistance on this academic work."

Submitted to:

Dr. Asif Ali Khan

Date:

1st June 2024

Department of Computer Systems Engineering

University of Engineering and Technology, Peshawar

Objectives:

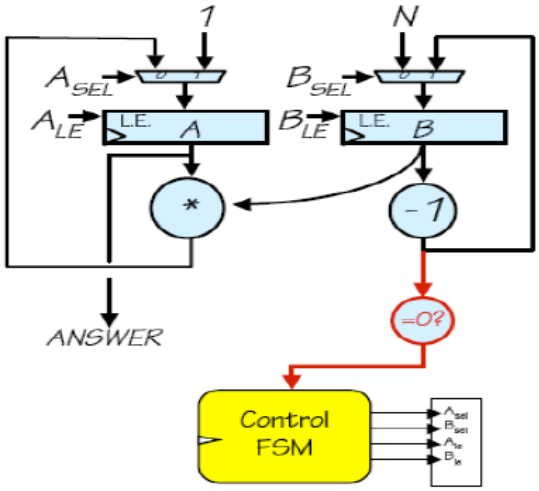
To implement a circuit that calculates factorial of a number.

Software used:

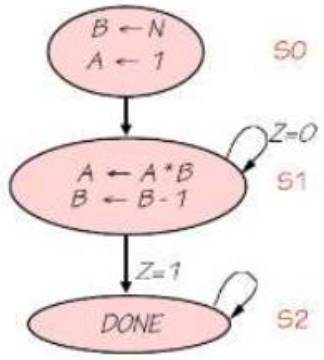
Xilinx ISE

Block Diagram:

The datapath for calculating the factorial of an "N" bit number is given below. The datapath is controlled by a Finite State Machine (FSM). FSM generates signals Asel, Ale, Bsel and Ble at the correct times to operate the datapath. Input to FSM is a signal "Z" when Z=0 means the operation of the machine is complete and ANSWER can be read. The STG is also given in the following diagram.



State Transition Diagram:



I/O Connection:

Connect the input N to switches so automatically N=8 bits and connect the ANSWER to 8 LEDs. Don't give input greater than 5 else the ANSWER will not 8 bit wide.

Lab Task 1:

Factorial Datapath + Control Path Simulation

Code:

```
21  module testbench();
22     reg CLK = 0;
23     reg [7:0]data_in;
24     reg start;
25     wire done;
26     wire [7:0]out;
27     wire[2:0] state;
28
29     wire eqz, regN_sel, regN_en, tempFact_en, tempFact_sel, calc_Product, Ld_DC, dec_DC, product_done;
30
31     wire [7:0]regN_out;
32     wire [7:0]Decrementer_Out;
33     controller cp(CLK, eqz, start, product_done, regN_sel, regN_en, tempFact_en, tempFact_sel, Ld_DC, dec
34     factorial_datapath dp(eqz,product_done, regN_sel, regN_en, tempFact_en, tempFact_sel, Ld_DC, dec_DC,
35
36     always # 5 CLK = ~CLK;
37
38     assign regN_out = dp.regN_out;
39     assign Decrementer_Out = dp.Decrementer_Out;
40     initial begin
41        $monitor("%d, RegN=%d,RegTemp=%d, TempProduct=%d", $time, dp.regN_out, dp.tempFact_out, dp.temp_pr
42        data_in = 8'd4;
43        start = 1;
44     end
45  endmodule
```

Design Summary | testbench.v

Figure 1.1 testbench module

```
21  module controller (clk, eqz, start, product_done, regN_sel, regN_en, tempFact_en, tempFact_s
22
23    input clk, eqz, start, product_done;
24
25    output reg regN_sel = 0 , regN_en  = 0 , tempFact_en  = 0 , tempFact_sel  = 0 , Ld_DC = 0,
26    output reg [2:0] state;
27
28    parameter S0=3'b000, S1=3'b001, S2=3'b010, S3=3'b011, S4=3'b100, S5=3'b101, S6=3'b111;
29
30    always@(posedge clk) begin
31      case (state)
32        S0:  if(start)
33               state <= S1;
34             else
35               state <= S0;
36
37        S1:  state <= S2;
38        S2:  state <= S3;
39        S3:  #2
40           if (product_done)
41              state <= S4;
42           else
```

Design Summary | testbench.v | controller.v

Figure 1.2.1: Control Path Code – Part 1

```
42          else
43             state <= S3;
44
45       S4:  state <= S5 ;
46
47       S5:  #2 if (eqz)
48             state <= S6;
49          else
50             state <= S2;
51       S6:  state <= S6;
52      default: state <= S0;
53     endcase
54   end
55
56   // combinational logic to generate datapath control signals
57   always @(*) begin
58     case (state)
59       S0:  begin regN_sel <= 0; regN_en <= 0; tempFact_en <= 0; tempFact_sel <= 0; Ld_DC = 0
60       S1:  begin regN_sel <= 1; regN_en <= 1; tempFact_en <= 1; tempFact_sel <= 1; Ld_DC = 0
61       S2:  begin regN_en <= 1; tempFact_en <= 1; Ld_DC = 0; dec_DC <= 0; calc_Product <= 0;
62       S3:  begin regN_sel <= 0; regN_en <= 0; tempFact_en <= 0; tempFact_sel <= 0; Ld_DC = 0
63       S4:  begin regN_sel <= 0; regN_en <= 0; tempFact_en <= 0; tempFact_sel <= 0; Ld_DC = 1
```

Figure 1.2.2: Control Path Code – Part 2

```
63       S4:  begin regN_sel <= 0; regN_en <= 0; tempFact_en <= 0; tempFact_sel <= 0; Ld_DC = 1
64       S5:  begin regN_sel <= 0; regN_en <= 0; tempFact_en <= 0; tempFact_sel <= 0; Ld_DC = 0
65       S6:  begin regN_sel <= 0; regN_en <= 0; tempFact_en <= 0; tempFact_sel <= 0; Ld_DC = 0
66     endcase
67   end
68 endmodule
```

Figure 1.2.3: Control Path Code – Part 3

```
21 module factorial_datapath (eqz, product_done, regN_sel, regN_en, tempFact_en, tempFact_sel, Ld_DC, dec_D
22     input regN_sel, regN_en, tempFact_en, tempFact_sel,Ld_DC, dec_DC, calc_Product, clk;
23     input [7:0] data_in;
24     output eqz;
25     output [7:0]out;
26     output product_done;
27
28     wire [7:0] MUX1_OUT, MUX2_OUT, regN_out, tempFact_out, Decrementer_Out;
29     wire [7:0] temp_product;
30
31     Mux2x1 mux1(.out(MUX1_OUT) , .I0(Decrementer_Out), .I1(data_in), .s(regN_sel));
32     PIPO1 A (regN_out, MUX1_OUT, regN_en, clk);
33
34     Mux2x1 mux2(.out(MUX2_OUT) , .I0(temp_product), .I1(1), .s(tempFact_sel));
35     PIPO2 P (tempFact_out, MUX2_OUT, tempFact_en, clrTemp, clk);
36
37     DECREMENTER DC (Decrementer_Out, regN_out, Ld_DC, dec_DC, clk);
38     EQZ COMP(eqz, regN_out);
39
40     multiplier mult(regN_out, tempFact_out, clk, calc_Product,product_done, temp_product);
41     PIPO1 D(out, temp_product, output_en, clk);
42
43 endmodule
```

Figure 1.3: Factorial Datapath Code

```
module Mux2x1(out , I0, I1, s);
    input [7:0]I0 , I1 ;
    input s;
    output [7:0]out;

    assign out = (s) ? I1 : I0;

endmodule
```

Figure 1.4: 2x1 Multiplexer Code

```
module PIPO1 (dout, din, ld, clk);
    input [7:0] din;
    input ld, clk;
    output reg [7:0] dout;

   always @ (posedge clk)
      if (ld)
         dout <= din;
endmodule
```

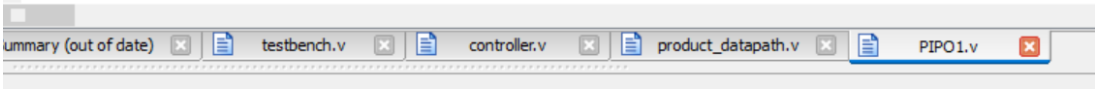


Figure 1.5: Parallel In Parallel Out (PIPO) Register 1 Code

```
module PIPO2 (dout, din, ld, clr, clk);
    input [7:0] din;
    input ld, clr, clk;
    output reg [7:0] dout = 8'd0;

    always @ (posedge clk)
         if (clr)
            dout <= 8'b0;
         else if (ld)
            dout <= din;

endmodule
```

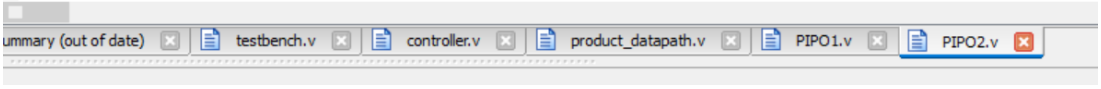


Figure 1.6: Parallel In Parallel Out (PIPO) Register 2 Code

```
21 module DECREMENTER (dout, din, ld, dec, clk);
22   input [7:0] din;
23   input ld, dec, clk;
24   output reg [7:0] dout;
25
26   always @(posedge clk) begin
27     if (ld)
28       dout <= din;
29     else if (dec)
30       dout <= dout - 1;
31   end
32 endmodule
```

esign Summary | testbench.v | controller.v | product_datapath.v | Mux2x1.v | PIPO1.v | CNTR.v

Figure 1.7: Decrementer Module Code

```
21 module EQZ (eqz, data);
22    input [7:0] data;
23     output eqz;
24     assign eqz = (data == 1);
25
26 endmodule
27
```

Figure 1.8: EQZ(Comparator) Module Code

```
21  module multiplier(
22      input [3:0] M,
23      input [3:0] Q,
24      input CLK,
25      input RST,
26      output reg product_done,
27      output reg [7:0] P = 8'd0
28  );
29      reg [3:0] A = 4'b0000;
30      reg [3:0] Q_reg;
31      reg [3:0] count = 4'b0000; // To count the number of shifts
32      reg C = 1'b0;
33
34      always @(posedge CLK) begin
35        if(~RST) begin
36           Q_reg = Q;
37           if(count > 0)
38              count = 0;
39        end
40
41        else begin
```

mary (out of date) | testbench.v | controller.v | product_datapath.v | PIPO1.v | PIPO2.v | multiplier.v

Figure 1.9.1: Unsigned 4-bit Multiplier Code – Part 1

```
42              if(count == 0) begin
43                  product_done = 0;
44                  Q_reg = Q;
45               end
46              if (count < 4) begin
47                  if (Q_reg[0]) begin
48                      {C,A} = A + M;
49                  end
50
51                  Q_reg = {A[0], Q_reg[3:1]}; // Shift right and include A's LSB in Q
52                  A = {C,A[3:1]};
53                  count = count + 1;
54              end
55        end
56          if (count == 4) begin
57              P <= {A, Q_reg};
58              product_done = 1;
59          end
60      end
61  endmodule
62
```

mary (out of date) | testbench.v | controller.v | product_datapath.v | PIPO1.v | PIPO2.v | multiplier.v

Figure 1.9.2: Unsigned 4-bit Multiplier Code – Part 2

Output:

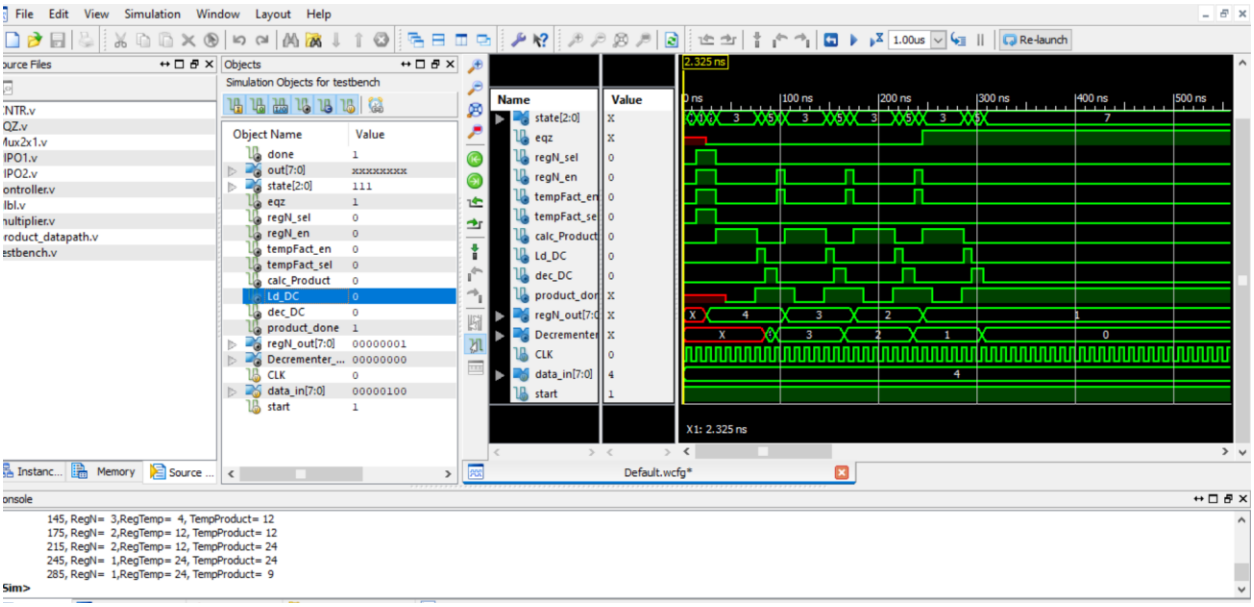


Figure 1.10: Output Waveform of Factorial Datapath and Control path using 4-bit Multiplier (Output is saved in RegTemp register)

Lab Task 2:

Product Datapath + Control Path Simulation

Code:

```
21  module testbench();
22     reg CLK = 0;
23     reg [3:0]data_in_A;
24     reg [3:0]data_in_B;
25     reg start;
26     reg [2:0] state = 3'b000;
27     wire done;
28     wire [7:0]out;
29     wire eqz, LdA, LdB, LdP, clrp;
30
31     controller cp(CLK, eqz, start, LdA, LdB, LdP, clrp, done, state);
32     product_datapath dp(eqz, LdA, LdB, LdP, clrp, data_in_A, data_in_B, CLK);
33
34     always # 5 CLK = ~CLK;
35     assign out = dp.Z;
36
37     initial begin
38        data_in_A = 4'd10;
39        data_in_B = 4'd10;
40        #10
41        start = 1;
42     end
```

Figure 2.1 testbench module

```
21  module controller (clk, eqz, start, LdA, LdB, LdP, clrp, done, state);
22
23    input clk, eqz, start;
24
25    output reg LdA = 0 , LdB  = 0 , LdP  = 0 , clrp  = 0 , done  = 0;
26    output reg [2:0] state;
27    //reg [2:0] state;
28    parameter S0=3'b000, S1=3'b001, S2=3'b010, S3=3'b011, S4=3'b100;
29
30    always@(posedge clk) begin
31      case (state)
32        S0:  if(start)
33               state <= S1;
34             else
35               state <= S0;
36        S1:  state <= S2;
37        S2:  state <= S3;
38        S3:  #2 if (eqz)
39               state <= S4;
40           else
41               state <= S3;
42        S4:  state <= S4;
43       default: state <= S0;
44      endcase
45    end
```

Design Summary (out of date) | testbench.v | controller.v

Figure 2.2.1: Product Control Path Code – Part 1

```
46
47    // combinational logic to generate datapath control signals
48    always @(*) begin
49      case (state)
50        S0:  begin LdA = 0; LdB = 0; LdP = 0; clrp = 0; done = 0; end
51        S1:  begin LdA = 1; LdB = 0; LdP = 0; clrp = 0; done = 0; end
52        S2:  begin LdA = 0; LdB = 1; LdP = 0; clrp = 0; done = 0; end
53        S3:  begin LdA = 0; LdB = 0; LdP = 1; clrp = 0; done = 0; end
54        S4:  begin LdA = 0; LdB = 0; LdP = 0; clrp = 1; done = 1; end
55      endcase
56    end
57
58  endmodule
59
60
```

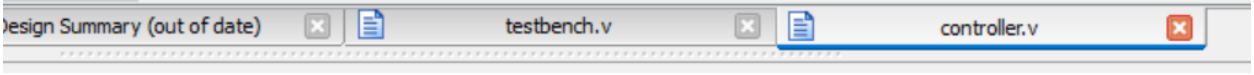


Figure 2.2.2: Product Control Path Code – Part 2

```
21  module product_datapath (eqz, LdA, LdB, Ldp, clrp, data_in_A, data_in_B, clk, out);
22      input LdA, LdB, Ldp, clrp, clk;
23      input [3:0] data_in_A;
24      input [3:0] data_in_B;
25      output eqz;
26      output [7:0]out;
27
28      wire [7:0] X, Y , Z;
29      PIPO1 A (X, data_in_A, LdA, clk);
30      PIPO2 B (Y, data_in_B, LdB, clrp, clk);
31
32     multiplier mult(X, Y, clk, Ldp, eqz, Z);
33
34     assign out = Z;
35
36  endmodule
37
38
```

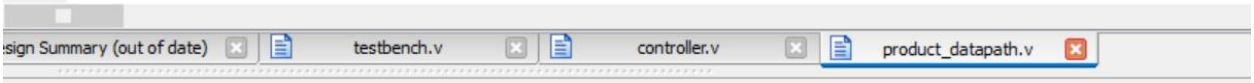


Figure 2.3: Product Datapath Code

Output:

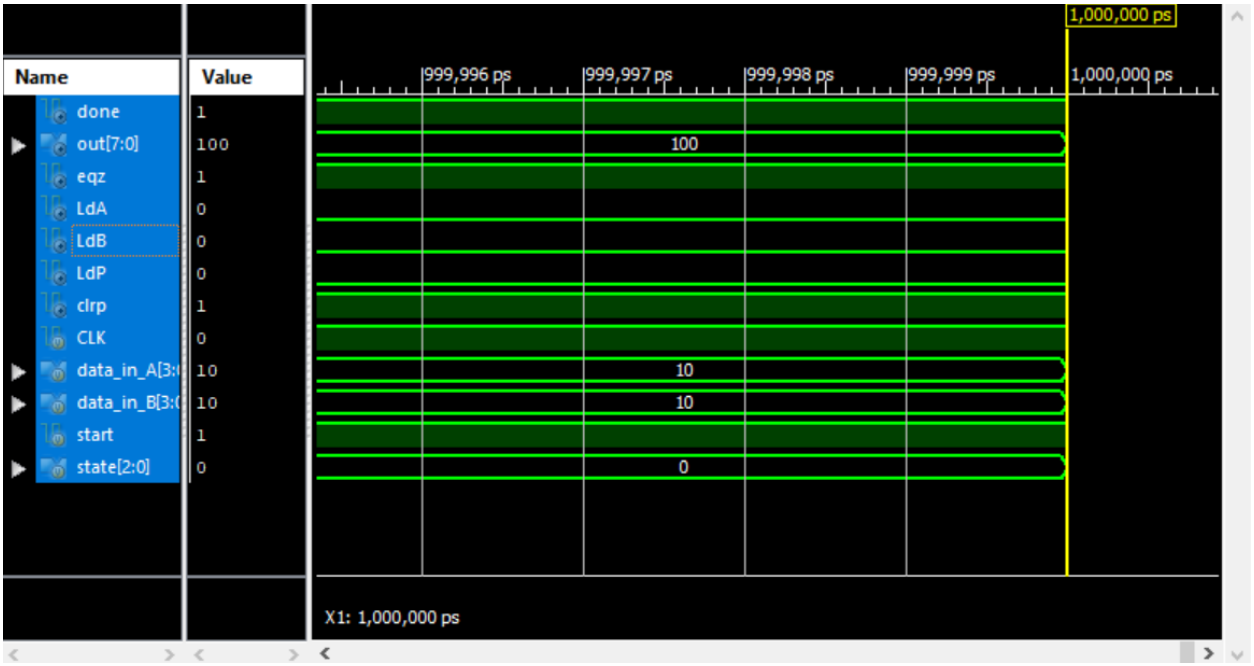


Figure 2.4: Output Waveform of Product Datapath and Control path using 4-bit Multiplier

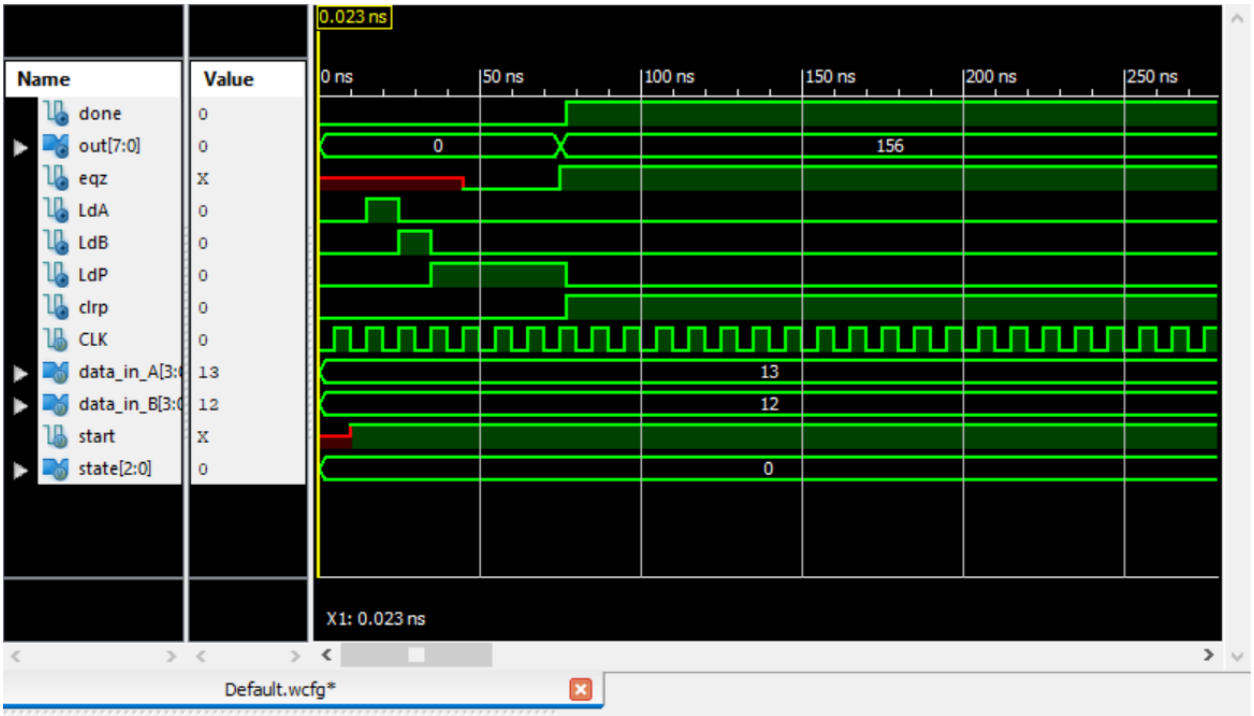


Figure 2.5: Output Waveform of Product Datapath and Control path using 4-bit Multiplier